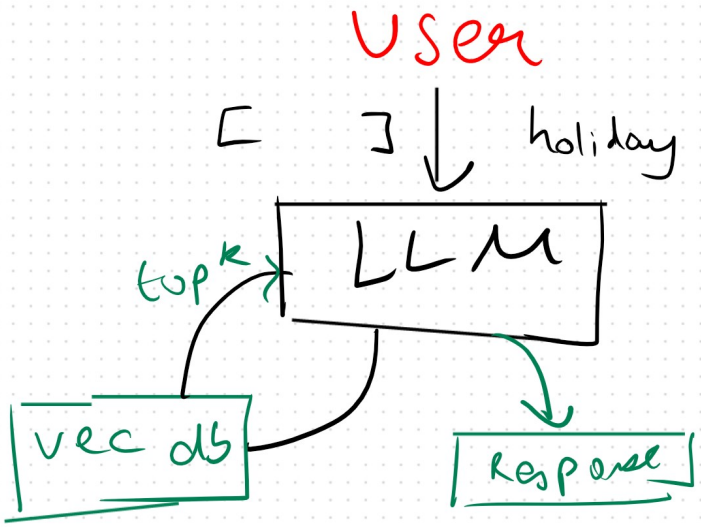
Revision :-

Rag → Retrieval Augmented generation

- Data ingestion
  - → Data load
  - → Data chunk
  - → Data embed
  - → Vector DB
- Data Retrieval



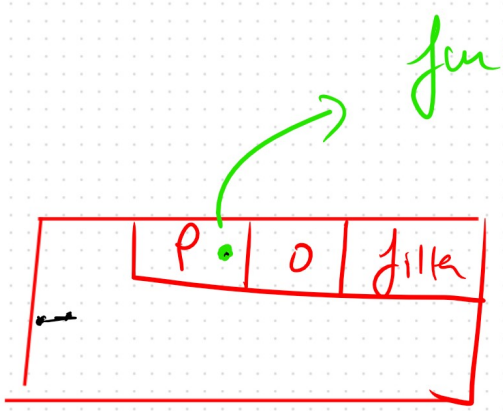
ReAct ✓

Create agent

Context window

- LLM → input → output
- Tool → py function
- Memory

Software:-

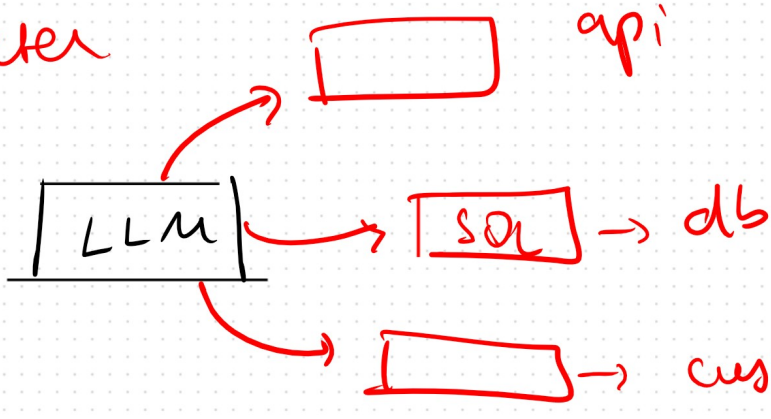



```
def list_products
    (- . )
    retur - - -
```



Rule Based, hard coded

parameter



api

def add(a, b):- query

"""
Docstring
"""

a, b, → c addd

c = a+b

return c

Memory :- → Hi, Divah → user

Hello → Ai

user :- coffee

Ai :- Recipe

Appending new msg in list of messages. → Buffer → episodic, semantic

user :- last quest

Ai → coffee

where we store this memory:

1. Inmemory
2. persistent → Dict → state
3. SQL → Postgress SQL

↳ langmem, Mem0.

Agent :- Context window

↳ Cutoff date

↳ LLM → 31st Oct

↳ 5 Oct

BARD

Openai → gpt 3 2023

google → BARD

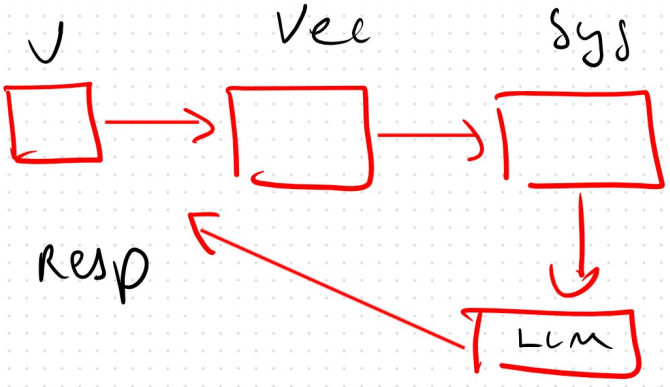
openai SDK → langchain

↓

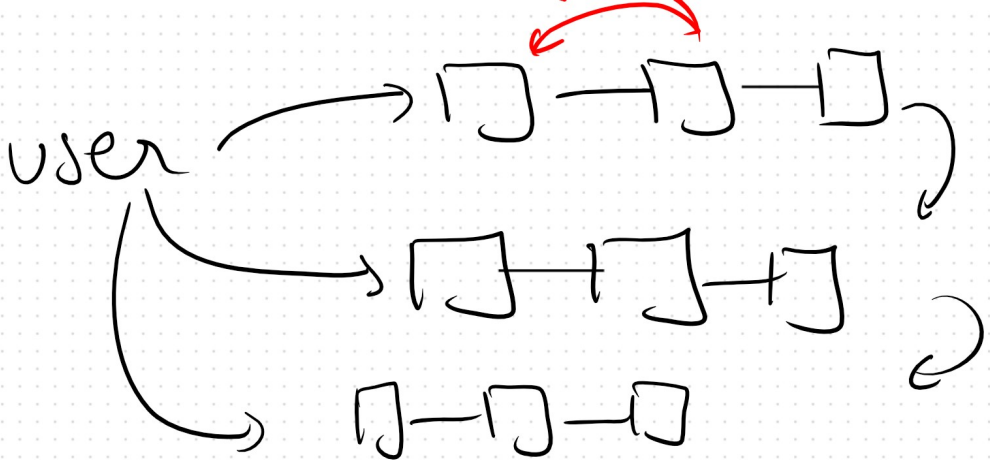
langgraph

Lang graph

Chain →



Dynamic



Context
Closs

modular level

control

↳ Research about dolo 650

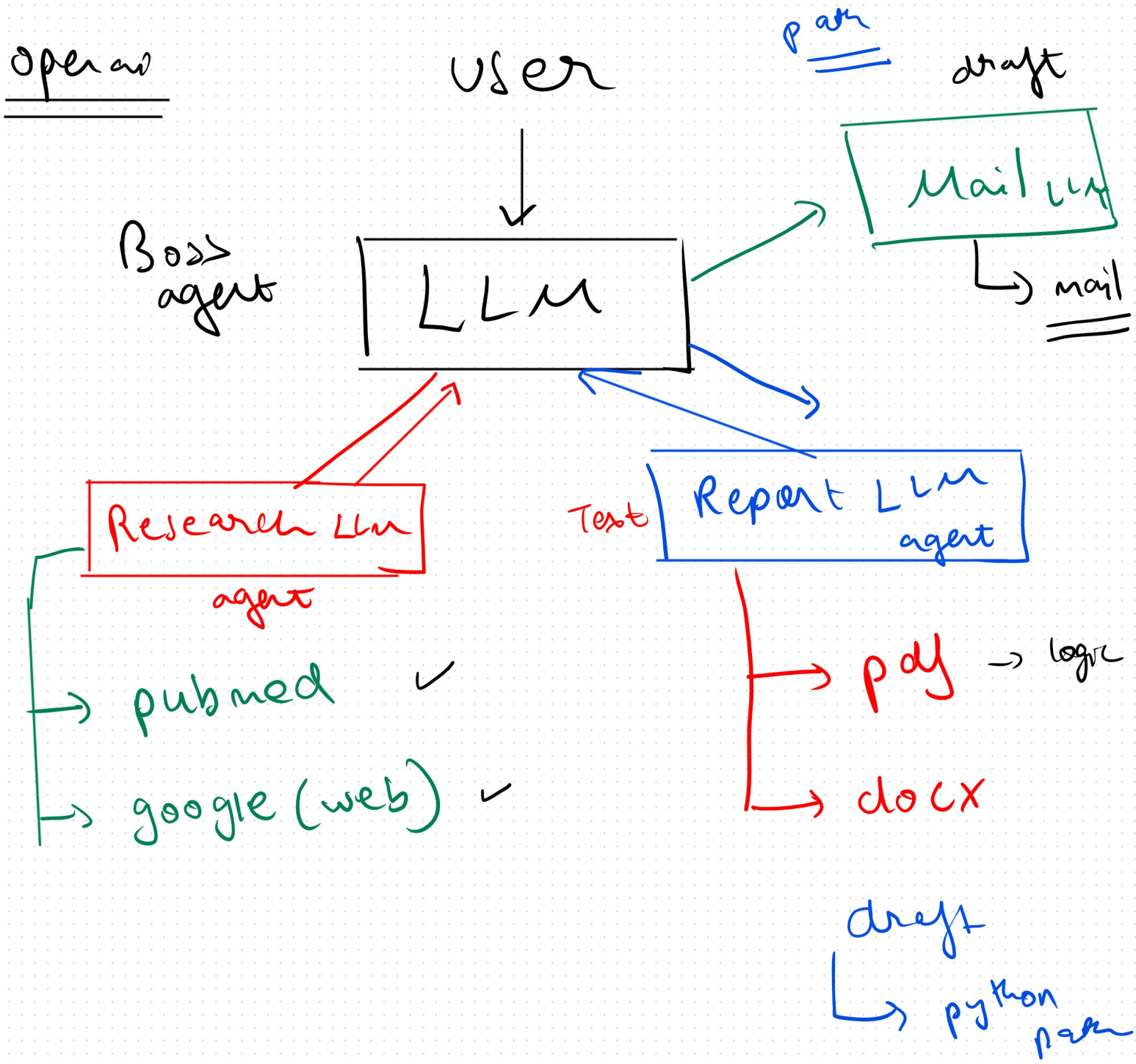
drawback

↳ Theory → web search

Once done prepare a report and mail to owner

agent → LLM

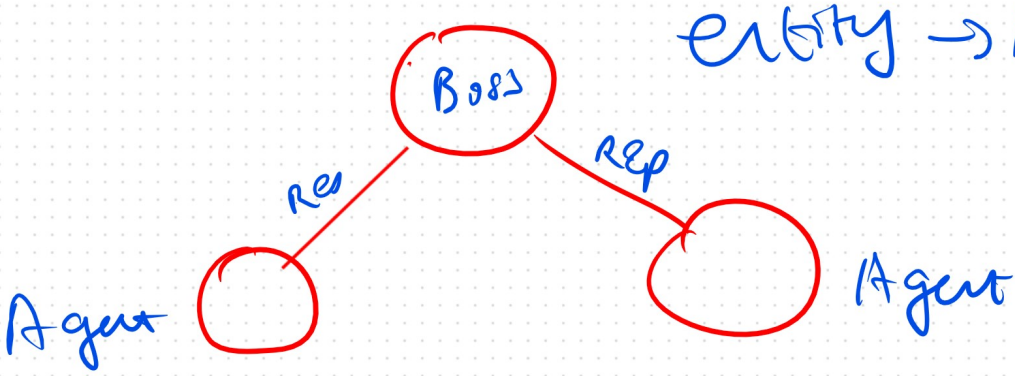
↳ tool


Operator
User
Boss agent
LLM
Mail LLM
draft
mail
Research LLM agent
Report LLM agent
Test
pubmed
google (web)
pdf
logs
docx
draft
python

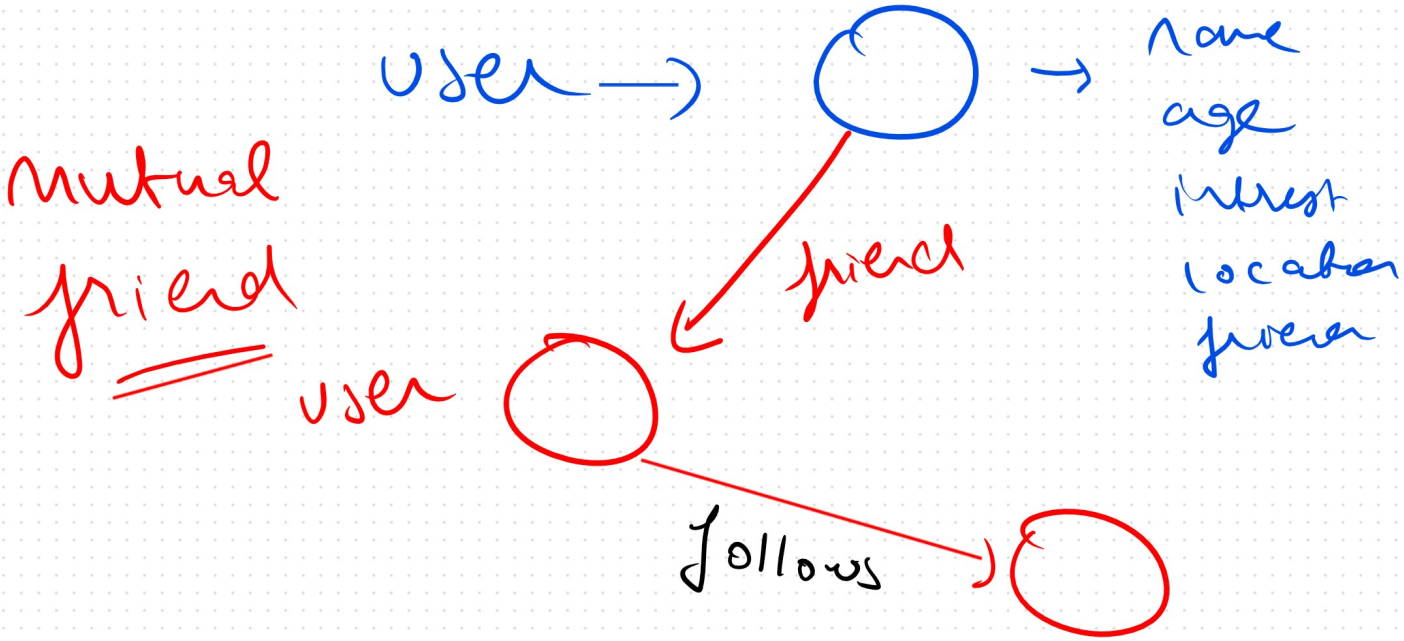
Langgraph → graph

nodes
edges

entity → Agent



facebook → user



State → Technical

graph → langgraph.

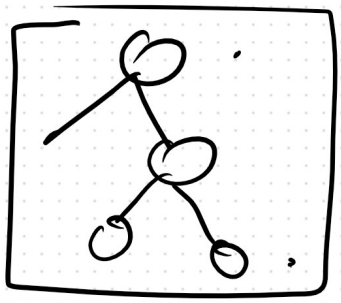
↳ State → context

a = StateGraph (State)

↓

class

↓

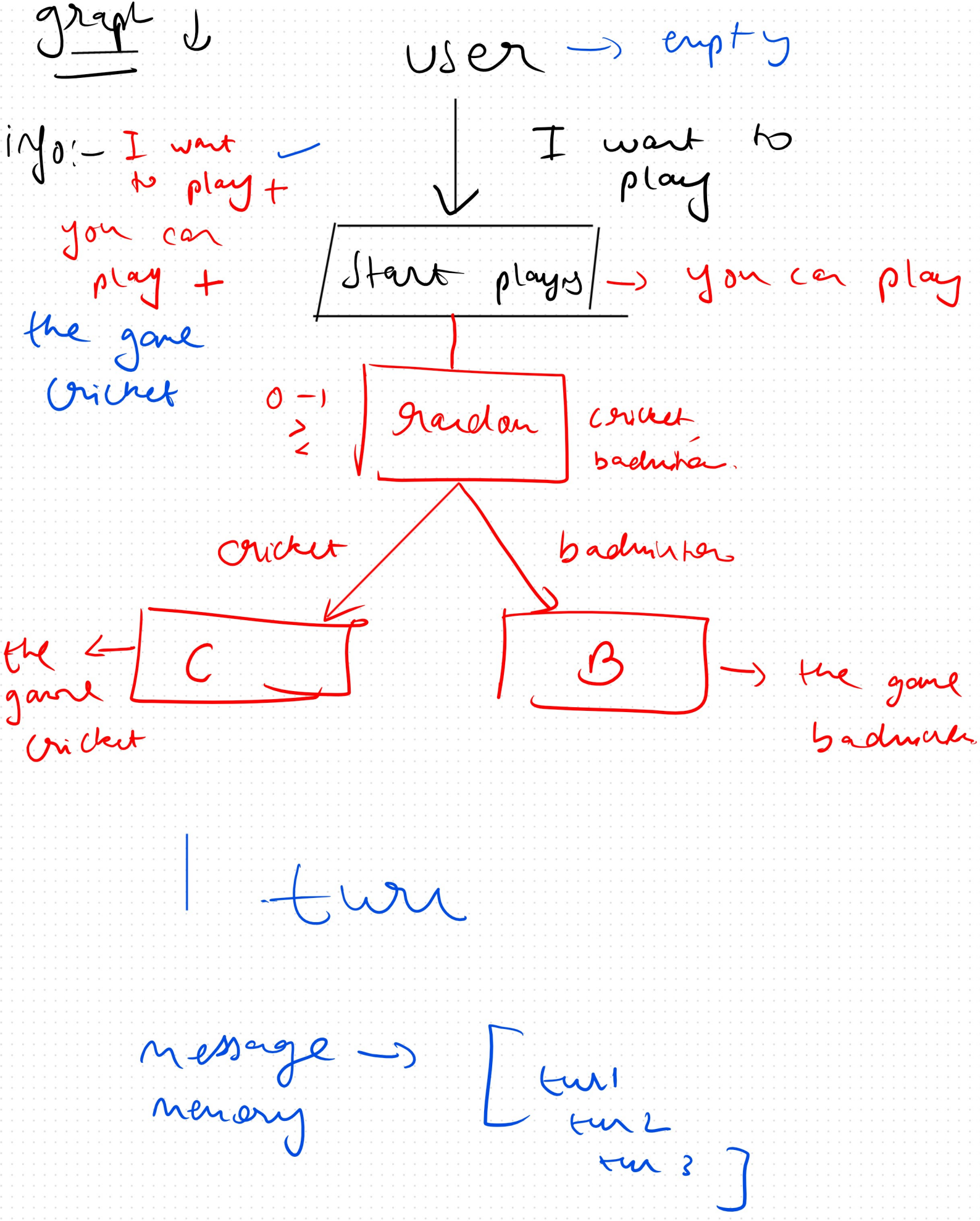


class

↳ structure defined

sportsClass:

→ graph_info : "str"



turn

message memory → [turn1 turn2 turn3]

| turn → [4 things.
↳ messages

[human message
system message
tool message
ai message] ←

running LLM ✓ invoke
or ↳ langchain, graph

agent → LLM
tool
memory.

[- user ✓ →
✓ ✓ sys-prompt → "
✓ ✓ tool →
✓ ai_msg] resp ←] 1 turn

→ Hi, make coffee → user

↳ ai → recipe → ai

[user.

ai]